

DET-015 | Outbound Connection to Rare Country / Exfiltration

ID: DET-015

MITRE ID:T1041

Data Source: DeviceNetworkEvents

Severity: Medium

Tactic:Exfiltration

Category:LATERAL MOVEMENT

ADX MAPPING LOGIC:

Mapping: BeginLat/BeginLon = 'destination coordinates (rare country)', DamageProperty = 'bytes sent proxy', State = 'source device'. We use a join to compare current vs historical baseline — simulating the rare-destination detection pattern.

OBJECTIVE: Practice join with a historical baseline — key operator for baselining and anomaly detection. Real Sentinel: join current connections with 30-day history to find IPs with no prior contact and high data transfer.

KQL QUERY (RUNNABLE ON ADX FREE TIER)

```
let RecentActivity = StormEvents
| where StartTime > ago(30d) // "current window"
| summarize
RecentConns = count(),
BytesSent = sum(DamageProperty),
DestList = make_set(State, 5)
by Source;
let HistoricalBaseline = StormEvents
| where StartTime between (ago(365d) .. ago(30d)) // "historical baseline"
| summarize HistoricalConns = count() by Source;
RecentActivity
| join kind=leftouter HistoricalBaseline on Source
| extend
IsRareDestination = isnull(HistoricalConns) or HistoricalConns < 3,
ExfilRisk = iff(BytesSent > 500000, "HIGH",
iff(BytesSent > 100000, "MEDIUM", "LOW"))
| where IsRareDestination == true
and BytesSent > 50000 // simulates >1MB threshold
| project Source,RecentConns,BytesSent,HistoricalConns,IsRareDestination,ExfilRisk,
DestList
| order by BytesSent desc
| take 20
```

INVESTIGATION STEPS

1. Lookup the remote IP in VirusTotal and AbuseIPDB.
2. Identify which process initiated the connection.
3. Review the amount of data transferred — large volumes = potential exfil.
4. Check if the domain/IP belongs to known C2 infrastructure.

RESPONSE ACTIONS

- Block the remote IP at the perimeter firewall.
- Isolate the device if C2 or exfiltration is confirmed.
- Review DLP logs for sensitive data transfer.
- Preserve network capture logs for forensic analysis.

Output from Above KQL Query:

```
1 let RecentActivity = StormEvents
2 | where StartTime > ago(30d) // "current window"
3 | summarize
4 RecentConns = count(),
5 BytesSent = sum(DamageProperty),
6 DestList = make_set(State, 5)
7 by Source;
8 let HistoricalBaseline = StormEvents
9 | where StartTime between (ago(365d) .. ago(30d)) // "historical baseline"
10 | summarize HistoricalConns = count() by Source;
11 RecentActivity
12 | join kind=leftouter HistoricalBaseline on Source
13 | extend
14 IsRareDestination = isnull(HistoricalConns) or HistoricalConns < 3,
15 ExfilRisk = iff(BytesSent > 500000, "HIGH",
16 iff(BytesSent > 100000, "MEDIUM", "LOW"))
17 | where IsRareDestination == true
18 and BytesSent > 50000 // simulates >1MB threshold
19 | project Source,RecentConns,BytesSent,HistoricalConns,
20 IsRareDestination,ExfilRisk,DestList
21 | order by BytesSent desc | take 20
```

Source	RecentConns	BytesSent	HistoricalConns	IsRareDestination	ExfilRisk	DestList
No Rows To Show						

KQL Detection Query in Sentinel:

DeviceNetworkEvents

```
| where TimeGenerated > ago(1h)
| where ActionType == "ConnectionSuccess"
| where RemoteIPType == "Public"
| summarize
ConnectionCount = count(),Ports = make_set(RemotePort),BytesSent = sum(SentBytes),
BytesRecv = sum(ReceivedBytes),FirstSeen = min(TimeGenerated),
LastSeen = max(TimeGenerated)
by DeviceName, RemoteIP, RemoteUrl
| join kind=leftouter (DeviceNetworkEvents
| where TimeGenerated > ago(30d)
| summarize HistoricalCount = count() by RemoteIP
) on RemoteIP
| where isnull(HistoricalCount) or HistoricalCount < 5
| where BytesSent > 1048576 // >1 MB sent
| project DeviceName, RemoteIP, RemoteUrl,ConnectionCount, BytesSent, BytesRecv,
Ports, FirstSeen, LastSeen
| order by BytesSent desc
```